



PROYECTO: SISTEMA WEB

Auditoría de Seguridad

Sistema Web Seguro

Sistemas de gestión de Seguridad de Sistemas Informáticos

SudoMotors

June Castro

Urko Horas

Aimar Larriba

Daniel Talmaci

Eneko Rodriguez

30/10/2025

Ingeniería Informática de Gestión y Sistemas de Información

ÍNDICE

1. INTRODUCCIÓN.....	3
2. AUDITORÍA DE SEGURIDAD (Web Insegura).....	4
2.1 ZAP.....	4
2.2 SQLmap.....	5
3. ANÁLISIS DE VULNERABILIDADES.....	7
1. Rotura de control de acceso.....	7
1.1 Acceso mediante URL.....	7
2. Fallos criptográficos.....	8
2.1 Mala gestión de claves.....	8
2.2 Almacenamiento y transmisión de datos en texto plano.....	9
3. Inyección.....	10
4. Diseño inseguro.....	11
5. Configuración de seguridad insuficiente.....	12
5.1 Cabecera Anti-Clickjacking (X-Frame-Options).....	12
5.2 Cabecera Content-Security-Policy (CSP).....	12
5.3 Cabecera X-Content-Type-Options.....	13
5.4 Tokens Anti-CSRF.....	13
5.5 Cookies sin atributos de seguridad (HttpOnly / SameSite).....	14
5.6 Divulgación de información en cabeceras HTTP (Server / X-Powered-By).....	14
6. Componentes vulnerables y obsoletos.....	14
6.1 Versiones de los distintos componentes utilizados en el sistema.....	15
7. Fallos de identificación y autenticación.....	15
7.1 Administración insegura.....	15
7.2 Gestión deficiente de sesiones.....	16
7.3 Contraseñas débiles.....	18
8. Fallos en la integridad de datos y software.....	18
9. Fallos en la monitorización de la seguridad.....	19
9.1. Monitorización de registros.....	19
4. AUDITORÍA DE SEGURIDAD (Web Segura).....	21
2.1 ZAP.....	21
2.2 SQLmap.....	21
5. CONCLUSIONES.....	22
6. BIBLIOGRAFIA.....	23

1. INTRODUCCIÓN

Esta parte del proyecto tiene como finalidad continuar asegurando la protección y resistencia de la página web que desarrollamos previamente.

Tras haber comprobado cuáles eran las vulnerabilidades de seguridad que exponían a nuestro sitio web mediante el uso de herramientas como ZAP y SQLmap, es el momento de ponerles solución.

Este proyecto no solo nos ayuda a fortalecer nuestra aplicación, sino que también nos permite ampliar de forma práctica nuestros conocimientos en materia de ciberseguridad. Gracias a esta experiencia, adquirimos habilidades que serán útiles para enfrentarnos a situaciones reales dentro del ámbito de la seguridad informática.

2. AUDITORÍA DE SEGURIDAD (Web Insegura)

2.1 ZAP

Para comenzar con esta auditoría de seguridad hemos utilizado el proxy ZAP. Mediante esta herramienta de código abierto hemos conseguido detectar diferentes vulnerabilidades en nuestro sistema web.

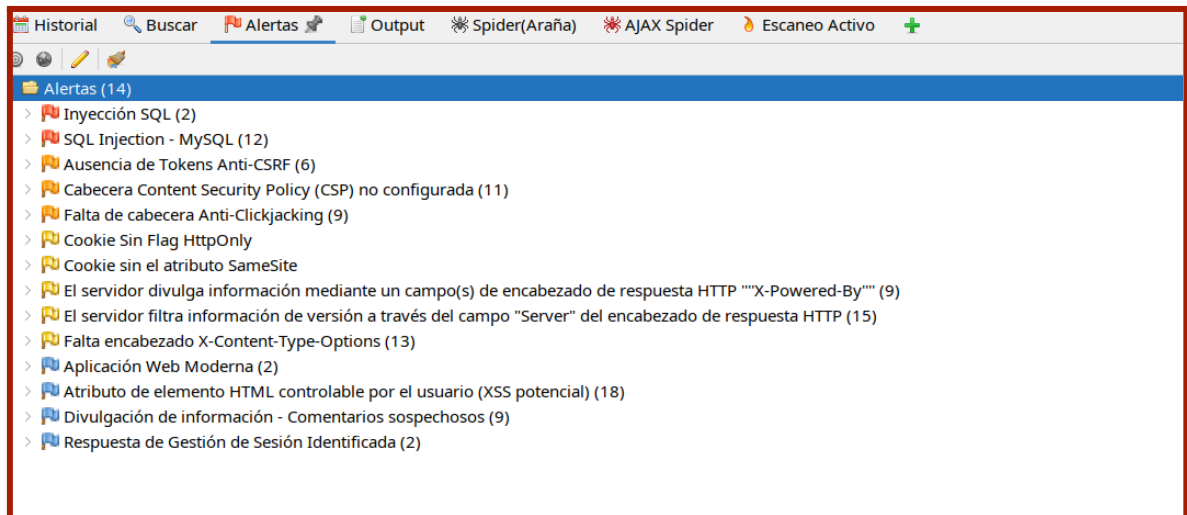


Figura 1

En la Figura 1 podemos ver el listado de errores y fallos en nuestra web, ordenados de mayor a menor riesgo.

Como podemos comprobar el más grave se refiere en cuanto a inyecciones SQL y por general falta de cookies y cabeceras.

2.2 SQLmap

También hemos utilizado SQLmap, otra herramienta de pentesting de código abierto, la cual detecta fallos de seguridad en cuanto a bases de datos.

```
daniel@laci:~/Downloads/sqlmapproject-sqlmap-03be590$ python3 sqlmap.py -u http://localhost:81/login.php --wizard --flu
sh-session

[1.9.10.5#dev]
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applica
ble local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 19:25:18 /2025-10-30/

[19:25:18] [INFO] starting wizard interface
POST data (--data) [Enter for None]:

[19:25:20] [WARNING] no GET and/or POST parameter(s) found for testing (e.g. GET parameter 'id' in 'http://www.site.com/vuln.php?id=1'). Will search for for
ms
Injection difficulty (--level/--risk). Please choose:
[1] Normal (default)
[2] Medium
[3] Hard
> 3
Enumeration (--banner/--current-user/etc). Please choose:
[1] Basic (default)
[2] Intermediate
[3] All
> 3
sqlmap is running, please wait..

[1/1] Form:
POST http://localhost:81/login.php
POST data: user=&contrasena=
do you want to test this form? [Y/n/q]
> Y
Edit POST data [default: user=&contrasena=] (Warning: blank fields detected): user=&contrasena=
do you want to fill blank fields with random values? [Y/n] Y
got a 302 redirect to 'http://localhost:81/items.php'. Do you want to follow? [Y/n] Y
redirect is a result of a POST request. Do you want to resend original POST data to a new location? [y/N] N
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y
```

Figura 2

```
Database: database
Table: USUARIO
[2 entries]
+-----+-----+-----+-----+-----+-----+-----+
| DNI | EMAIL | NOMBRE | TELEFONO | USERNAME | APELLIDOS | CONTRASENA | F_NACIMIENTO |
+-----+-----+-----+-----+-----+-----+-----+
| 12345678-Z | altorji@gmail.com | Aitor | 668252000 | altorjiti | Jimenez+Jimenez | RonCola300 | 2002-10-12 |
| 22770213-Y | juneji@gmail.com | June | 667925412 | junecastro | Alvarez+Jimenez | Vodkalimon200 | 2001-02-16 |
+-----+-----+-----+-----+-----+-----+-----+

Database: database
Table: VEHICULO
[3 entries]
+-----+-----+-----+-----+-----+
| AÑO | KMS | MARCA | MODELO | MATRICULA |
+-----+-----+-----+-----+-----+
| 2025 | 235 | Ford | Focus | 3326+IOP |
| 2001 | 89985 | Ferrari | Spider | 6255+XDD |
| 2018 | 205623 | Kia | Sportage | 7895+TYU |
+-----+-----+-----+-----+-----+

[*] ending @ 19:22:47 /2025-10-30/

daniel@laci:~/Downloads/sqlmapproject-sqlmap-03be590$
```

Figura 3

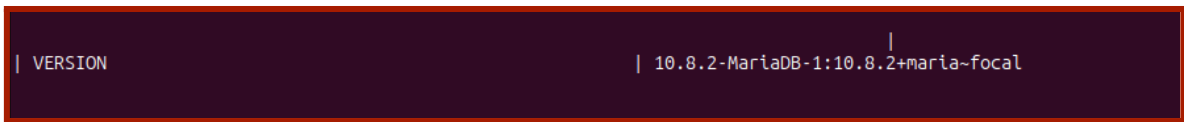


Figura 4

En la primera figura desplegamos la herramienta de SQLmap. En la Figura 2 podemos observar como la herramienta ha podido obtener toda la información de nuestra base de datos con las contraseñas incluidas. Además de mostrar también las versiones de los diferentes servicios que utiliza la página web

3. ANÁLISIS DE VULNERABILIDADES

1. Rotura de control de acceso

Una gestión de acceso ineficiente o, directamente, carecer de ella como en nuestro caso hace que cualquier usuario del sistema, aunque no debería de tener permisos para ello, pueda acceder a secciones del sistema destinadas a la administración o ciertos usuarios, como es el caso de las modificación de datos personales. Esta carencia de mecanismos que controlen los permisos de los usuarios representa una fragilidad en la seguridad del sistema, la cual es el fallo más común del OWASP Top 10.

1.1 Acceso mediante URL

Así pues, se pueden modificar o robar datos de otros usuarios, existe riesgo de incumplir el principio de mínimo privilegio y de violar la confidencialidad del sistema. además de acceder a APIs sin control para los métodos HTTP POST, PUT, y DELETE.

De esta manera, si se quieren evitar estas vulnerabilidades se deberá negar el acceso a todos los recursos privados, centralizar el control de acceso y validar el acceso para cada recurso solicitado, todo ello aplicando el principio de denegación por defecto. Además, es recomendable registrar el intento de múltiples intentos de acceso fallidos, por si se trata de un ataque.

SOLUCIÓN:

Se ha añadido el parámetro ROLE a los usuarios, de tal forma que todos los usuarios tendrán el rol “user”, a excepción de administrador que tendrá “admin”. Mediante la implementación de un nuevo php encargado del control del login y del rol del usuario, ahora se controla de forma centralizada el acceso a contenido restringido; en este caso, el add, modify y delete de vehículos. Las acciones show_user y modify_user, por su parte, son exclusivas del usuario dueño del perfil.

```

<?php
session_start();

function requireLogin() {
    if (empty($_SESSION['USERNAME'])) {
        header("Location: login.php");
        exit;
    }
}

function requireAdmin() {
    requireLogin();
    if ($_SESSION['ROLE'] !== 'admin') {
        http_response_code(403);
        die("Acceso denegado: se requieren privilegios de administrador.");
    }
}
}

```

Figura 5

```

include 'includes/access_control.php';

// -----
// CONTROL DE ACCESO
// Solo los administradores pueden añadir vehículos
// -----
requireLogin();
requireAdmin();

```

Figura 6

Acceso denegado: se requieren privilegios de administrador.

Figura 7

2. Fallos criptográficos

Los fallos en los métodos criptográficos pueden provocar una protección insuficiente o nula de los datos sensibles, tanto en tránsito como en reposo. Estas vulnerabilidades suelen deberse a algoritmos débiles, claves inseguras o configuraciones erróneas.

2.1 Mala gestión de claves

En nuestro sistema, no existe ningún tipo de cifrado mediante claves y su correspondiente gestión. Así pues, la confidencialidad, integridad y disponibilidad de la información pueden quedar seriamente comprometidas en caso de ataque.

Primero que todo, la ausencia de algoritmos robustos de generación de claves y de conexiones seguras puede dar lugar a ataques Man in the middle o robo de información. Además, cuando se implementen claves en el sistema, es necesario asegurar que estas tienen una rotación y revocación de claves correcta, ya que, en caso contrario, puede dar lugar a que estas sean comprometidas.

En la situación actual del sistema, los datos del sistema no poseen seguridad ninguna, poniendo así en duda la fiabilidad del sistema. Por consiguiente, es necesario adoptar distintas medidas que garanticen un uso y gestión de claves como conexiones cifradas seguras y confiables.

De esta forma, es imprescindible la implementación de claves generadas con una alta entropía, para evitar usar claves débiles, y de conexiones seguras al sistema web mediante SSL/TLS (https). Asimismo, es indispensable asegurar que solo el personal autorizado tiene acceso a las claves del sistema. En cuanto a la rotación de las mismas, es recomendable no alargar su uso durante un largo tiempo y evitar el uso de claves caducadas. Por último, a la hora de revocarlas, no deben ser eliminadas completamente, si no almacenarse de forma segura por si es necesario su uso en el futuro.

2.2 Almacenamiento y transmisión de datos en texto plano

Guardar los diferentes datos del sistema, especialmente las contraseñas de los usuarios, en texto plano o sin las debidas medidas de protección puede derivar en una pérdida masiva de información o en su posible modificación por terceros no autorizados para ello.

Esta falla en la seguridad afecta a todo el sistema web desarrollado, dado que actualmente no se cifran de ninguna forma los datos. Asimismo, en cuanto a las contraseñas, esta ausencia de cifrado permite a un atacante obtener las credenciales de otro usuario y acceder a sus cuentas, poniendo así en riesgo su confidencialidad y vulnerando la normativa de protección de datos.

Con el objetivo de mitigar al máximo lo mencionado anteriormente, es necesario clasificar los datos según su nivel sensibilidad, cifrar toda la información mediante protocolos seguros, como TLS con Forward Secrecy, además de reducir el almacenaje de datos sensibles todo lo posible. En cuanto a la gestión de contraseñas se recomienda almacenarlas utilizando funciones hash seguras combinadas con salts de la mayor entropía posible.

SOLUCIÓN:

Para proteger los datos en tránsito se cifran usando HTTPS. Para ello, hemos generado un certificado autofirmado y hemos abierto el puerto 443 de tal forma que la página redirige automáticamente a HTTPS.

```
ports:
- "81:80"
- "443:443"
```

Figura 8

Además para proteger los datos en reposo se protegen con hash, como ya se ha explicado en otros puntos.

De esta manera, se terminan de cifrar los datos que puedan estar expuestos en la aplicación.

```
$hashed_password = password_hash($password, PASSWORD_DEFAULT);
```

Figura 9

3. Inyección

Si no se comprueban los datos de entrada, existe la posibilidad de que estos sean interpretados como código por el sistema y, de esta forma, que un atacante modifique contenido no accesible o ejecute scripts maliciosos en el navegador de otros usuarios, conocido como Cross-Site Scripting.

Actualmente en el sistema, los datos introducidos por el usuario no se filtran o sanean de ninguna forma antes de ser utilizados en el servidor. En consecuencia, existe una vulnerabilidad importante ante ataques mediante inyección.

De esta forma, en todas las partes de nuestro sistema que se reciben datos externos, como pueden ser formularios, cabeceras etc. existe este peligro. Como consecuencia, la integridad y disponibilidad de la información del sistema, el comportamiento y la privacidad de las cuentas puede quedar comprometida. Esto se debe a que las consultas que se realizan en el sistema son consultas dinámicas, es decir, no están parametrizadas.

Con el fin de evitar este tipo de ataques, es indispensable validar y sanear todos los inputs del usuario; esto es, eliminar todos los caracteres especiales posibles. Además, usar controles como LIMIT para evitar la pérdida masiva de datos y APIs seguras o, en su defecto, llamadas parametrizadas u ORM para evitar que los datos se inserten directamente en sentencias SQL.

SOLUCION:

Se valida el parámetro y, si es necesario, se sanea para evitar ataques mediante inyección a través de la URL añadiendo código al principio del php, tal y como aparece a continuación.

```
$matricula = strtoupper(trim($_GET['matricula']));
if (!preg_match('/^[0-9]{4}\s?[A-Z]{3}$/', $matricula)) {
    die("Matrícula no válida.");
}
```

Figura 10

Asimismo, de igual manera que con el parámetro de la URL, se sanean de forma preliminar los parámetros de entrada introducidos por el usuario, para posteriormente poder ser validado el formato en el js correspondiente.

```
$new_matricula = strtoupper(trim($_POST['matricula'] ?? ''));
$new_marca = htmlspecialchars(trim($_POST['marca'] ?? ''), ENT_QUOTES, 'UTF-8');
$new_modelo = htmlspecialchars(trim($_POST['modelo'] ?? ''), ENT_QUOTES, 'UTF-8');
$new_ano = filter_var($_POST['ano'] ?? '', FILTER_VALIDATE_INT);
$new_kms = filter_var($_POST['kms'] ?? '', FILTER_VALIDATE_INT);
```

Figura 11

Por otra parte, con el fin de evitar la inyección SQL en las consultas, el código de estas se ha modificado para que las consultas ahora sean parametrizadas.

```
$stmt = $conn->prepare("SELECT * FROM VEHICULO WHERE MATRICULA=?");
$stmt->bind_param("s", $matricula);
$stmt->execute();
$query = $stmt->get_result();
```

Figura 12

Por último, para evitar el XSS (Cross-Site Scripting) se añade la siguiente cabecera en todo el sistema.

```
header("X-XSS-Protection: 1; mode=block");
```

Figura 13

4. Diseño inseguro

Durante el análisis se identificó que el sistema no fue desarrollado bajo un enfoque de seguridad desde el diseño (Security by Design). Varias funciones críticas carecen de controles básicos, lo que permite que las vulnerabilidades se propaguen a otras partes de la aplicación.

En el análisis ZAP se detectó falta de validación en el lado del servidor, reflejada en alertas de inyección SQL y XSS. Además, la ausencia de tokens Anti-CSRF, la falta de cabeceras como Content-Security-Policy, X-Frame-Options y X-Content-Type-Options, así como cookies sin atributos HttpOnly ni SameSite, confirman deficiencias en la configuración de seguridad.

También se observó divulgación de información a través de los encabezados X-Powered-By y Server, lo que evidencia un diseño inseguro y la ausencia de medidas preventivas básicas. Estas vulnerabilidades muestran que la aplicación prioriza la

funcionalidad sobre la seguridad, comprometiendo la confidencialidad e integridad de los datos.

SOLUCIÓN:

La solución para este problema está en general repartida en las soluciones de otros problemas ya que la falta de estos token, cabeceras y cookies, generaban otras alertas. Por lo que al resolverlas, el diseño ha pasado a ser seguro o, al menos, más seguro de lo que era.

Como resumen, se han añadido los tokens Anti-CSRF en los archivos donde el usuario introduce información en formularios para asegurar que sea el usuario quien lo hace.

Además, se han añadido las cabeceras y cookies necesarias para eliminar las alertas y aumentar la seguridad de nuestra web. Estas cabeceras y cookies están explicadas en las soluciones que lo requieren expresamente.

```
<?php
function verificar_csrf() {
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        if (!isset($_POST['csrf_token']) || $_POST['csrf_token'] !== $_SESSION['csrf_token']) {
            die("Error: solicitud no válida (CSRF detectado).");
        }
    }
}
```

Figura 14

5. Configuración de seguridad insuficiente

La configuración de seguridad incorrecta o insuficiente se refiere a ajustes o parámetros de seguridad que no están adecuadamente configurados para proteger un sistema, aplicación, red o servicio. Puede haber varias áreas en las que la configuración de seguridad puede ser insuficiente o incorrecta, lo que podría dejar un sistema vulnerable a amenazas y ataques.

5.1 Cabecera Anti-Clickjacking (X-Frame-Options)

El análisis de ZAP muestra la ausencia de la cabecera X-Frame-Options. Esta cabecera es fundamental para prevenir ataques de tipo clickjacking, en los que un atacante incrusta el sitio web dentro de un iframe malicioso con el fin de engañar al usuario para que realice acciones no deseadas.

En nuestra página web se manifiesta observando que al seleccionar una petición (por ejemplo, index.php), en el apartado Response Headers, no aparece la cabecera X-Frame-Options.

SOLUCIÓN:

Hemos añadido la cabecera X-Frame-Options: DENY en el archivo includes/head.php para evitar que la página se cargue dentro de iframes externos. Con esto prevenimos ataques de

tipo clickjacking, asegurando que el sitio solo pueda mostrarse directamente en el navegador del usuario.

```
header("X-Frame-Options: DENY");
header("X-Content-Type-Options: nosniff");
header("Content-Security-Policy: default-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; script-src 'self'; object-src 'none';");
```

Figura 15

5.2 Cabecera Content-Security-Policy (CSP)

La cabecera Content-Security-Policy (CSP) no está configurada. Esta política limita los recursos que el navegador puede cargar, y su ausencia deja el sistema vulnerable a XSS e inyecciones de código.

En la consola del navegador (pestaña Network → Headers), al inspeccionar cualquier respuesta HTTP, no aparece la cabecera Content-Security-Policy. Esto significa que cualquier script externo puede ejecutarse sin restricciones.

SOLUCIÓN:

Hemos implementado la cabecera Content-Security-Policy en includes/head.php, restringiendo la carga de recursos a nuestro propio dominio (default-src 'self') y evitando la ejecución de scripts externos. Esto protege la web frente a ataques XSS e inyecciones de código.

```
header("X-Frame-Options: DENY");
header("X-Content-Type-Options: nosniff");
header("Content-Security-Policy: default-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; script-src 'self'; object-src 'none';");
```

Figura 16

5.3 Cabecera X-Content-Type-Options

Esta cabecera previene que el navegador interprete archivos con un tipo MIME distinto al declarado, lo que evita ejecución de contenido inesperado (por ejemplo, un script con extensión .jpg). Su ausencia puede derivar en ejecución de código malicioso.

Al inspeccionar las cabeceras HTTP de cualquier recurso (por ejemplo, index.php o login.php), no aparece X-Content-Type-Options. Esto se observa en el panel “Headers” de las herramientas del navegador o dentro del reporte de ZAP.

SOLUCIÓN:

Hemos configurado la cabecera X-Content-Type-Options: nosniff para impedir que el navegador interprete archivos con tipos MIME incorrectos. De esta forma se evita la ejecución accidental de código malicioso a partir de archivos no seguros.

```
header("X-Frame-Options: DENY");
header("X-Content-Type-Options: nosniff");
header("Content-Security-Policy: default-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; script-src 'self'; object-src 'none';");
```

Figura 17

5.4 Tokens Anti-CSRF

No existen tokens de verificación CSRF en formularios sensibles (login, alta o modificación de datos). Esto permite ataques de Cross-Site Request Forgery, donde un atacante puede enviar peticiones no autorizadas en nombre del usuario autenticado.

Formularios como login.php, editar.php o insertar.php no incluyen ningún campo oculto que contenga un token CSRF. En el código fuente HTML de dichos formularios no aparece ningún valor dinámico o hash vinculado a la sesión. ZAP genera la alerta “Ausencia de Tokens Anti-CSRF”.

SOLUCIÓN:

Hemos creado un token CSRF único por sesión en el archivo includes/security.php, que se incluye como campo oculto en todos los formularios POST y se valida al procesar las peticiones. Así evitamos que un atacante pueda realizar acciones en nombre del usuario.

```
<?php
function verificar_csrf() {
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        if (!isset($_POST['csrf_token']) || $_POST['csrf_token'] !== $_SESSION['csrf_token']) {
            die("Error: solicitud no válida (CSRF detectado).");
        }
    }
}
?>
```

Figura 18

5.5 Cookies sin atributos de seguridad (HttpOnly / SameSite)

Las cookies utilizadas para mantener la sesión no incluyen los atributos de seguridad HttpOnly ni SameSite. Esto facilita ataques de robo de sesión mediante JavaScript o peticiones cruzadas.

ZAP reporta las alertas: “Cookie sin flag HttpOnly” y “Cookie sin el atributo SameSite”.

SOLUCIÓN:

Hemos modificado la configuración de sesión en includes/head.php para que las cookies incluyan los atributos HttpOnly y SameSite=Strict. Esto impide el acceso a la cookie mediante JavaScript y bloquea su envío en peticiones cruzadas.

```
// Solo configuramos las cookies si la sesión NO está activa
if (session_status() === PHP_SESSION_NONE) {
    session_set_cookie_params([
        'httponly' => true,
        'secure' => false,    // Cambiar a true si usamos HTTPS
        'samesite' => 'Strict'
    ]);
    session_start();
}

// Token CSRF (si no existe, se crea)
if (!isset($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
```

Figura 19

5.6 Divulgación de información en cabeceras HTTP (Server / X-Powered-By)

El servidor revela información sobre la tecnología utilizada, como PHP, Apache o su versión exacta, a través de cabeceras HTTP (Server, X-Powered-By). Esto facilita ataques dirigidos contra versiones concretas del software.

En el reporte de ZAP aparecen las alertas “El servidor divulga información mediante el campo X-Powered-By” y “El servidor filtra información de versión a través del campo Server”.

SOLUCIÓN:

Hemos arreglado la fuga de información del servidor configurando las cabeceras HTTP desde PHP. Los encabezados “Server” y “X-Powered-By” son eliminados o sobrescritos antes del envío de la respuesta en los archivos principales, lo que disminuye la exposición de detalles sobre la tecnología utilizada. Esta forma de hacer impide a los posibles atacantes identificar versiones específicas del software y disminuye el riesgo de uso posterior de vulnerabilidades conocidas.

6. Componentes vulnerables y obsoletos

El uso de componentes vulnerables u obsoletos es una de las causas más comunes de fallos de seguridad en aplicaciones web y sistemas informáticos. Este tipo de vulnerabilidad ocurre cuando un sitio o aplicación utiliza librerías, frameworks, módulos o dependencias de software que contienen errores conocidos, versiones sin soporte o configuraciones inseguras. Es decir, causan vulnerabilidades debido a su antigüedad o a la falta de actualizaciones.

6.1 Versiones de los distintos componentes utilizados en el sistema

Es necesario mantener actualizados los sistemas operativos y los componentes utilizados por el sistema web, de esta manera asegurándonos de contener las últimas actualizaciones de seguridad y parches.

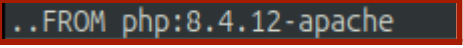
Nuestra página web utiliza servicios, tales como PHP o MariaDB, algo anticuados, debido a que el proyecto está desarrollado a partir de una base proporcionada. Esto puede llevar a

no tener todas las funcionalidades de seguridad disponibles tal y como se ha comentado anteriormente.

SOLUCIÓN:

Para solucionarlo hemos actualizado las versiones de los servicios que utiliza nuestro sistema, de esta manera asegurándonos que el sistema contiene todas las actualizaciones de seguridad actuales.

PHP: 7.2.2 pasa a 8.4.2



```
FROM php:8.4.12-apache
```

Figura 20

MariaDB: 10.8.2 pasa a 12.0.2



```
db:
  image: mariadb:12.0.2
```

Figura 21

7. Fallos de identificación y autenticación

Los fallos de identificación y autenticación son vulnerabilidades que afectan a los mecanismos encargados de reconocer y verificar la identidad de los usuarios dentro de un sistema. Estos mecanismos tienen como objetivo garantizar que solo las personas autorizadas puedan acceder a determinados recursos o información.

7.1 Administración insegura

En este caso de proyecto, al realizar nuestro sistema web en base a un proyecto ya iniciado, la contraseña del administrador para identificarse en PhpMyAdmin es muy insegura. A parte de ser conocida por las demás personas las cuales han utilizado la misma base para su proyecto.

También existe la posibilidad de que cualquier persona elimine los vehículos del sistema desde PhpMyAdmin. Además nuestra web no tiene implementada la funcionalidad de Administrador como tal, ya que todos los usuarios tienen los mismos permisos para modificar o eliminar datos del sistema. Esto supone que cualquier usuario del sistema pueda hacer lo que quiera con los datos del sistema, cosa que no conviene en un sistema.

SOLUCIÓN:

En este caso hemos tenido que modificar el archivo docker-compose.yml para poder cambiar la contraseña del administrador de la base de datos, tanto PhpMyAdmin como MariaDB.

Para el cambio hemos decidido utilizar un archivo .env y variables en el docker-compose ya que el usuario que levanta el servidor no siempre tiene por que conocer la contraseña y

el usuario de estos servicios. En este caso el .env va dentro de la carpeta del proyecto al ser un trabajo evaluable.

```
db:
  image: mariadb:12.0.2
  restart: always
  volumes:
    - ./mysql:/var/lib/mysql
  environment:
    MYSQL_ROOT_PASSWORD: root
    MYSQL_USER: ${MYSQL_USER}
    MYSQL_PASSWORD: ${MYSQL_PASSWD}
    MYSQL_DATABASE: ${MYSQL_DB}
  ports:
    - "8889:3306"

phpmyadmin:
  image: phpmyadmin/phpmyadmin:latest
  links:
    - db
  ports:
    - 8890:80
  environment:
    MYSQL_USER: ${MYSQL_USER}
    MYSQL_PASSWORD: ${MYSQL_PASSWD}
    MYSQL_DATABASE: ${MYSQL_DB}
```

Figura 22

En la Figura podemos ver como ahora el usuario, la contraseña y el nombre de la base de datos son “desconocidos” y se extraen del archivo .env.

Nota: Si al iniciar el sistema la contraseña no ha cambiado, es por que en la carpeta mysql no se han actualizado las variables. Para solucionarlo, simplemente se ha de eliminar la carpeta y volver a levantar el sistema, de esta manera recreando la carpeta con las nuevas variables.

7.2 Gestión deficiente de sesiones

Las sesiones son algo importante a tener en cuenta a la hora identificación y autenticación; ya que si esta queda abierta se pueden llegar a producir ataques de tipo de secuestro de sesión en las cuales un atacante toma el control de una sesión de usuario activa.

En nuestro sistema, a pesar de estar ya implementada desde el principio la opción para cerrar la sesión, no se asegura que si un usuario se olvida de cerrar la sesión, esta finaliza. Para ello, existen mecanismos que aseguran que después de un tiempo especificado la sesión finaliza.

El hecho de que la sesión quede abierta supone que pueda haber acceso no autorizado por terceros, que se robe el identificador de sesión o que los datos sensibles de un usuario queden expuestos.

SOLUCIÓN:

La primera parte de este fallo en cuantos gestión de sesiones sería el ser capaz de cerrar la sesión dentro del sistema. Esta opción ya estaba implementada en la primera versión de

nuestro programa ya que no nos imaginamos que esto sería una aportación a la seguridad de la web, simplemente es una opción que las páginas web usuales suelen tener.



Figura 23

En cuanto al hecho de que la sesión quede abierta, para asegurarnos de que queda cerrada hemos añadido un a cookie de sesión, la que nos asegura que al cerrar el navegador, la sesión quedará cerrada.

```
// Comienza la sesión configurando a su vez las cookies de sesión
session_start([
    'cookie_lifetime' => 0,           // La sesión se cierra cuando se cierra el navegador.
    'cookie_path' => '/',
    'cookie_secure' => true,          // La cookie se envia sobre conexiones HTTPS.
    'cookie_httponly' => true,        // La cookie solo es accesible a través de HTTP.
    'cookie_samesite' => 'Lax',       // Define la política de SameSite.
]);
```

Figura 24

El código de la Figura 23 se encuentra en index.php, de esta manera nada más abrir la página web al iniciar la sesión se configuran las cookies.

7.3 Contraseñas débiles

Las contraseñas débiles son contraseñas que son fáciles de adivinar o que se utilizan en muchos sistemas diferentes. Esto puede permitir a un atacante obtener acceso no autorizado a un sistema o aplicación. Las contraseñas débiles hacen que los ataques por fuerza bruta sean más sencillos de realizar de esta manera obteniendo las claves de otros usuarios para iniciar sesión en el sistema

SOLUCIÓN:

Para evitar que los usuarios puedan utilizar contraseñas débiles, hemos implementado una función adicional en la que se pide al usuario unos requisitos para asegurar unos requisitos mínimos de seguridad en cuanto a su contraseña.

```
function validPsswd(psswd) { // Comprueba que la contraseña es segura
    return (/^(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*(),.?":{}|<>]).{8,}$/.test(psswd.trim()));
}
```

Figura 25

8. Fallos en la integridad de datos y software

Durante el análisis se identificó que el sistema no garantiza la integridad de los datos ni del software. No existen controles que aseguren que la información almacenada o modificada por el usuario sea válida y no haya sido alterada de forma maliciosa.

Se detectó falta de validación de datos, una gestión de permisos insuficiente y ausencia de comprobaciones de integridad en operaciones críticas, lo que permite que los usuarios puedan modificar información sin restricciones ni verificaciones adecuadas.

Además, algunos procesos de manipulación de datos no registran cambios ni realizan controles de coherencia, lo que aumenta el riesgo de corrupción o alteración no autorizada del contenido.

Estas debilidades exponen al sistema a ataques como inyección SQL, Cross-Site Scripting (XSS), y al uso de cookies sin atributos seguros (HttpOnly y SameSite), evidenciando un nivel de riesgo alto para la integridad y confidencialidad de la información.

Los atacantes podrían aprovechar los datos expuestos por la aplicación para ejecutar ataques, ya sea manipulando información desde el propio sistema o introduciendo datos maliciosos que comprometan su seguridad.

SOLUCIÓN:

En cuanto a los permisos, sólo un administrador podrá introducir datos nuevos a la web (sin tener en cuenta los registros), y solo un administrador podrá modificar los datos que ya existen en la web.

```
function requireAdmin() {
    requireLogin();
    if ($_SESSION['ROLE'] !== 'admin') {
        http_response_code(403);
        die("Acceso denegado: se requieren privilegios de administrador.");
    }
}
```

Figura 26

```
requireLogin();
requireAdmin();
```

Figura 27

Además se han implementado las cabeceras y cookies necesarias para evitar que los datos sean accesibles cuando no deben de serlo. Estas medidas quedan explicadas más en detalle en los puntos que requieren dichas medidas de forma más explícita.

9. Fallos en la monitorización de la seguridad

Los fallos en la monitorización de la seguridad se refieren a deficiencias o problemas en los sistemas y procesos diseñados para vigilar y evaluar la seguridad de una red, sistema informático, aplicación o entorno en general.

9.1. Monitorización de registros

En nuestro sistema no existe una monitorización adecuada de los registros (logs). Actualmente solo se comprueba si un usuario introduce correctamente sus datos para iniciar sesión, pero no se registran ni se analizan los intentos de acceso.

Esto implica que podríamos sufrir un ataque de fuerza bruta sin darnos cuenta, ya que un atacante podría probar un gran número de contraseñas hasta encontrar una válida.

La solución a este problema sería hacer un implementar un sistema de registro que haga un seguimiento de los intentos que hace un único usuario para iniciar sesión y además proporcionaremos un número limitado de intentos. Si se detecta una actividad sospechosa (demasiados intentos de acceso fallido) permite activar alertas y bloquear ataques antes de que causen daño, además de bloquear temporalmente la cuenta o la dirección IP.

SOLUCIÓN:

Primero de todo, necesitamos llevar la cuenta o guardar cuántas veces seguidas falla un usuario al iniciar sesión.

Para ello, hemos creado una nueva tabla en nuestra base de datos llamada *LOGIN_INTENTOS*, la cual almacena la información de cada intento de acceso fallido realizado por un usuario o dirección IP.

```
CREATE TABLE IF NOT EXISTS `LOGIN_INTENTOS` (
  `ID` INT AUTO_INCREMENT PRIMARY KEY,
  `USERNAME` VARCHAR(50),
  `IP_ADDRESS` VARCHAR(45),
  `INTENTOS` INT DEFAULT 0,
  `LAST_ATTEMPT` DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Figura 28

Esta tabla nos permite detectar cuando se han superado el límite de intentos (5) y aplicar medidas de seguridad que en este caso trata de no permitirle realizar un nuevo intento a esa IP durante 1 minuto.

Además para evitar confusiones y que el usuario sea consciente del tiempo que falta para un nuevo intento hemos incorporado una cuenta atrás del tiempo que queda:

Demasiados intentos fallidos. Espera 54 segundos.

Figura 29

Una vez pasados los 60 segundos, los intentos se reinician, y el usuario vuelve a tener 5 oportunidades, antes de que vuelva a ser bloqueado.

Por otro lado, es importante llevar un registro de todos los intentos, tanto exitosos como fallidos, para así poder revisar si ha habido alguna actividad sospechosa. Para ello hemos creado otra tabla nueva llamada *LOGIN_LOGS* que guarda esta información

```
CREATE TABLE IF NOT EXISTS `LOGIN_LOGS` (
  `ID` INT AUTO_INCREMENT PRIMARY KEY,
  `USERNAME` VARCHAR(50),
  `IP_ADDRESS` VARCHAR(45),
  `FECHA` DATETIME DEFAULT CURRENT_TIMESTAMP,
  `RESULTADO` ENUM('exito','fallo') NOT NULL,
  `NAVEGADOR` TEXT,
  `DETALLES` TEXTS|
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Figura 30

El administrador del sistema es el responsable de revisar periódicamente los datos registrados en esta tabla, con el objetivo de detectar patrones anómalos o comportamientos que puedan indicar un intento de ataque o un uso indebido del sistema.

4. AUDITORÍA DE SEGURIDAD (Web Segura)

Después de ver todos los errores de seguridad que nuestra web tenía y arreglarlos uno a uno, ahora realizaremos tal y como hicimos antes de que la web fuera segura una auditoría utilizando las mismas herramientas.

2.1 ZAP

Primero hemos realizado una auditoría utilizando la herramienta de ZAP y en la siguiente Figura se puede ver el listado de posibles fallos de seguridad.

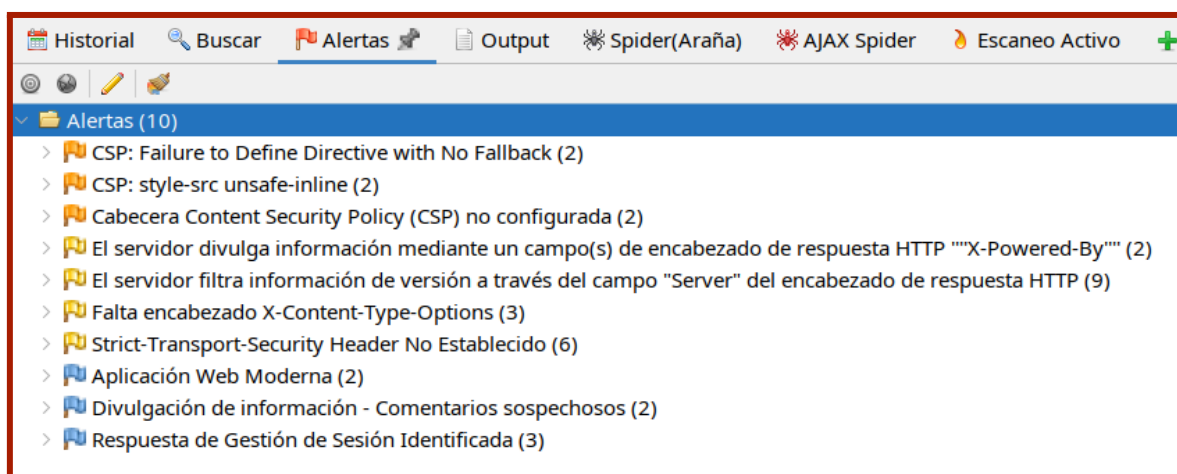


Figura 31

En comparación con la primera auditoría, ya no se presenta ningún fallo grave que existía anteriormente en nuestra web. Lo que supone un gran cambio.

5. CONCLUSIONES

A lo largo de este proyecto hemos logrado transformar un sistema web inicialmente vulnerable en una aplicación mucho más segura y robusta frente a distintos tipos de ataque. Mediante la auditoría de seguridad realizada con herramientas como ZAP y SQLmap, identificamos las principales debilidades del sistema, y posteriormente aplicamos medidas efectivas para mitigarlas.

Entre las mejoras implementadas destacan la incorporación de cabeceras de seguridad HTTP (como CSP, X-Frame-Options o X-Content-Type-Options), la introducción de tokens Anti-CSRF, el uso de contraseñas hasheadas, la configuración segura de cookies y la gestión de sesiones, así como la implementación de un control de roles para restringir el acceso según privilegios. Además, se añadieron mecanismos de monitorización mediante registros de intentos de inicio de sesión, fortaleciendo la detección de posibles ataques.

Estas medidas no solo corrigen las vulnerabilidades detectadas, sino que también consolidan una base sólida para el desarrollo seguro de aplicaciones futuras.

En definitiva, este proyecto nos ha permitido comprender la importancia de aplicar principios de seguridad desde las fases iniciales del desarrollo, adoptando un enfoque de Security by Design que garantice la integridad, confidencialidad y disponibilidad de la información en todo momento.

6. BIBLIOGRAFIA

- OpenAI (ChatGPT-5). <https://chatgpt.com/>
- Manual PHP <https://www.php.net/manual/en/index.php>
- OWASP. (2021). Informe de Vulnerabilidades. OWASP. <https://owasp.org/www-project-top-ten/>
- ZAP. Documentación de ZAP. <https://www.zaproxy.org/getting-started/>
- SQLmap. Documentación de SQLmap. <https://sqlmap.org/>